

COMPLETE TECHNICAL REFERENCE

BuildDoctor

A GitHub App that reads failing CI logs, works out what broke, and comments on the commit that caused it. Every file, every external service, every decision — and the questions an interviewer will ask.

Live at builddoctor.onrender.com

App github.com/apps/builddoctor-ci

Source github.com/aeturijayasimhayadav2004/builddoctor

~9,600 lines · 14 phases · 3 live installations

Verified end-to-end 2026-08-28

SECTION 00

What is in this document

1. The thirty-second pitch

What it does, in the words you would use out loud.

2. The lifecycle of one failing build

Ten steps from GitHub's webhook to a comment on your commit.

3. Architecture

The four processes, and why they are arranged this way.

4. Every file, and why it exists

All 19 Python modules, the React app, and the config.

5. External services

Six things it does not own, what each is for, and what was rejected.

6. The data model

Two tables, and the one column that governs access.

7. The decisions worth defending

The parts an interviewer will push on.

8. Interview questions, with answers

30 questions, easy to hard, and what each is really testing.

9. Proof it works

A real run, captured today.

HOW TO USE THIS

Sections 1–3 are what you say in the first two minutes. Section 7 is what separates a candidate who built something from one who copied a tutorial — every item is a decision with a reason and a rejected alternative. Section 8 is rehearsal.

Do not memorise section 4. Know the shape of it; look up the detail. An interviewer is testing whether you can explain a system, not whether you can recite it.

SECTION 01

The thirty-second pitch

SAY THIS

"BuildDoctor is a GitHub App that watches repositories for failing CI runs. When a build fails, it downloads the log, cuts it down to the lines around the error, sends that to an LLM along with the code diff that triggered the run, and posts a comment on the commit explaining what broke. It also classifies each failure into one of three lanes — and if the failure looks transient, it re-runs the job by itself."

"It remembers. Every diagnosis is stored with a vector embedding of the log, so when a similar failure happens again it passes the previous diagnosis to the model as a hint."

The three lanes

LANE	MEANING	ACTION TAKEN
informational	A real code or config error a human must fix	Post an explanation, stop
safe_auto_fix	Transient — network blip, flaky test, runner hiccup	Re-run the failed job automatically
needs_review	Ambiguous, or touches something sensitive	Flag it for a person

THE HONEST BOUNDARY — SAY THIS BEFORE YOU ARE ASKED

It **never writes code**. It does not push commits, open pull requests, or edit files, and the App's permissions do not allow it to — `contents` is read. The only write actions are: post a comment, add a label, re-run a job. That limit is deliberate; an agent that edits your code on a guess is a liability, not a feature.

Why this is a real project and not a toy

- **It has real users.** Three live installations, one belonging to somebody who is not the author.
- **It is measured.** A 24-case golden set scores the classifier at 91.7% overall, 95.2% on unambiguous cases. Regressions are caught by re-running it.
- **It fails closed.** Every access decision defaults to "no" when evidence is missing.
- **It survived its own security incident.** A secret leaked in Phase 10; the response is written up in the README rather than hidden.

SECTION 02

The lifecycle of one failing build

This is the answer to "walk me through what happens." Ten steps.

1 GitHub sends a webhook

A `workflow_run` event with action `completed` hits `POST /webhook`. GitHub expects a reply within **10 seconds**.

2 Verify the signature

HMAC-SHA256 over the raw body using `WEBHOOK_SECRET`, compared with `hmac.compare_digest`. Fails → 401, nothing else runs. This happens *first*, before parsing.

3 Check the gate

Look up `installations.is_allowed` for this installation id. Closed → log and drop. No log downloaded, no model called, no money spent.

4 Reply 200 immediately, work in the background

The real work is handed to a background task so GitHub's 10-second clock is never at risk. A slow LLM must not mark the App as a broken integration.

5 Mint a scoped token

`app_auth.py` signs a JWT with the App's private key, trades it for an **installation access token** scoped to that one installation, expiring in 1 hour. Cached in memory until then.

6 Gather evidence

List the failed jobs → download their logs → cut each to **5 lines before / 10 after** each `##[error]` marker → fetch the diff (PR diff, or commit vs parent).

7 Search memory

Embed the excerpt into 384 dimensions, then a pgvector cosine search over past diagnoses. A match above **0.90** similarity becomes a hint for the model.

8 Classify

The LangGraph `classify` node calls the LLM with the excerpt, the diff, and any past match — asking for a diagnosis *and* a lane, as structured JSON.

9 Route and act

The graph routes on the lane to an action node, which calls the **MCP server** over HTTP: post a comment, add a label, or re-run the job.

10 Record

Write a row to `diagnoses` — repo, run id, lane, excerpt, diff summary, diagnosis text, installation id, and the embedding, so this failure is findable next time.

MEASURED, NOT ESTIMATED

End to end, on a warm server: ~**18 seconds** from the build failing to the comment appearing.
Measured today — see section 9.

SECTION 03

Architecture

Four logical components

COMPONENT	JOB	WHERE IT RUNS
Web app	Webhooks, OAuth sign-in, the read API, the admin routes	FastAPI, one Render web service
Agent graph	Classify → route → act, as a state machine	LangGraph, in-process
MCP server	The four tools that actually write to GitHub	Mounted at <code>/internal/mcp</code> in the same process
Dashboard	Every diagnosis, and the approve/revoke panel	React SPA, served at <code>/dashboard/</code>

THE QUESTION THIS DESIGN INVITES

"Why is the MCP server a separate service at all if it runs in the same process?"

Because the **boundary is the point**, not the process count. Every action that writes to GitHub is reachable exactly one way — through a described, typed tool call. The agent cannot reach `github_client.py` directly. That means the complete list of things this agent can do to your repository is four function signatures, and you can read all four in a minute.

Under Docker they genuinely are separate containers. On Render they were merged into one process by `server_combined.py` — because the free tier gives one service, and paying for architectural purity when the boundary is enforced in code anyway is a bad trade.

The four MCP tools — the complete write surface

TOOL	DOES
<code>post_pr_comment</code>	Comment on a pull request
<code>post_commit_comment</code>	Comment on a commit
<code>add_pr_label</code>	Add a label to a PR
<code>rerun_workflow_job</code>	Re-run a failed job

THE RE-RUN LOOP GUARD

An agent that re-runs failed jobs can loop forever: re-run fails → webhook → re-run. The guard uses GitHub's own `run_attempt` field — BuildDoctor only ever re-runs attempt 1. Attempt 2 failing is classified but never re-run.

Crucially this is **stateless**: the counter is GitHub's, not a variable in memory that a restart would reset. That is the difference between a guard and a hope.

The MCP endpoint is public but unusable

The MCP server is mounted inside a public web service, so its URL is reachable. It refuses anyway:

`mcp_server.py` checks the `Host` header against an allowlist and answers **421 Misdirected Request** on a mismatch. A request through Render's router carries `Host: builddoctor.onrender.com`, which is not on the list. The app's own call carries `Host: localhost:10000`, which is.

Verified both directions before deploying: 421 for a foreign Host, successful handshake listing all four tools over loopback.

SECTION 04

Every file, and why it exists

Core pipeline — the request path

FILE	LINES	SIGNIFICANCE
main.py	1311	The front door. All web routes, HMAC signature verification, the access gate, evidence gathering, and every installation-lifecycle webhook handler (install, uninstall, repos added/removed, repo made private). The auto-approval rule lives here in <code>public_only_verdict()</code> .
graph.py	332	The agent. A LangGraph state machine: <code>classify → route → act</code> . Routing is a pure function of the lane, so the control flow is inspectable rather than emergent.
diagnose.py	521	The LLM call. Builds the prompt, demands structured JSON (diagnosis + lane + rationale), validates the response, and falls back to <code>informational</code> if the model returns something unusable. Model id lives at line 25.
log_excerpt.py	67	Token economy. A CI log can be megabytes; a context window is not. Finds <code>##[error]</code> markers and keeps 5 lines before, 10 after. Small file, large effect on cost and accuracy.
github_client.py	387	Every GitHub API call. Logs, diffs, comments, labels, re-runs. Every function takes an <code>installation_id</code> — there is no ambient credential to accidentally use.
mcp_server.py	295	The write surface. Four tools, plus DNS-rebinding protection via the Host allowlist.
mcp_client.py	219	How <code>graph.py</code> reaches those tools over HTTP. Also where retry lives.

Memory — the part that makes it interesting

FILE	LINES	SIGNIFICANCE
embeddings.py	150	Turns a log excerpt into 384 numbers using <code>all-MiniLM-L6-v2</code> , running locally. No API call, no per-embedding cost, no third party sees the log twice.
memory.py	150	"Has this failed before?" A pgvector cosine search. The 0.90 threshold at line 71 was derived from measurement, not guessed — see section 7.
backfill_embeddings.py	110	One-time: embedded the rows written before memory existed, so history was searchable rather than starting from zero.
migrate_jsonl.py	139	One-time: imported the original flat-file <code>diagnoses.jsonl</code> into Postgres.

Identity, access, and the dashboard

FILE	LINES	SIGNIFICANCE
app_auth.py	422	The App's own identity. Signs a JWT with the private key, trades it for a 1-hour installation token, caches it. This replaced a personal access token entirely — see section 7.

FILE	LINES	SIGNIFICANCE
user_auth.py	418	A person's identity . GitHub OAuth, and a signed session cookie. The GitHub user token is thrown away at the end of the callback and never stored.
db.py	976	Every line of SQL in the project . Two tables, the migration runner, and the access-gate queries. No ORM magic scattered elsewhere.
dashboard.py	325	The read-only <code>/api</code> routes: <code>/me</code> , <code>/stats</code> , <code>/diagnoses</code> . No writes anywhere in the file.
admin.py	221	The owner-only approve/revoke routes. Who counts as owner is read from GitHub's <code>GET /app</code> — not from a config file that could drift.
notify.py	124	One line of text to a Discord or Slack webhook. Fire-and-forget; cannot raise, because it runs inside a webhook handler on GitHub's clock.
server_combined.py	130	Mounts the app and the MCP server into one process, so Render's single free service can host both.
preflight.py	366	Asks all six external dependencies whether they are alive, checks every installation's gate state, and confirms the protected routes still return 401. Run before demoing.

SECTION 04 CONTINUED

The frontend and the eval suite

React dashboard — frontend/src/

FILE	LINES	SIGNIFICANCE
App.tsx	312	Layout and the three empty states — signed out, no installations, waiting for approval. Each is a different problem with a different fix, so each gets different words.
api.ts	275	Every fetch. <code>API_BASE</code> defaults to same-origin, which is what makes the session cookie work.
DiagnosisTable.tsx	244	The main table, with lane filtering.
AdminPanel.tsx	181	Approve/revoke, visible only to the owner. Shows <i>how</i> each install was approved — <code>auto · public</code> vs by hand.
StatCards.tsx	147	Totals by lane.
RefreshAccess.tsx	67	Re-runs OAuth to refresh a stale installation list. A button rather than automatic, because automatic would require storing the user token.
PendingApproval.tsx	72	The "waiting for approval" state — a correct state, not an error, and worded that way.
SignIn / Skeleton / LaneBadge / MemoryCell / icons / format	~430	Sign-in screen, loading skeletons, lane pills, the memory-match cell, inline SVG icons, date and text formatting.

The eval suite — eval/

FILE	LINES	SIGNIFICANCE
golden_set.json	24 cases	The measuring stick. Four groups: synthetic (8), historical (6), middle-zone (6), live-crosscheck (4). Each has a known-correct lane and a written rationale.
run_eval.py	1029	Runs the golden set against the real classifier — not a copy. It calls <code>graph.classify</code> and <code>memory.search_past_failures</code> , but never routes to the action nodes, so it cannot post or re-run anything.
findings.md	–	What the numbers <i>mean</i> . Written by hand; survives a re-run.

FILE	LINES	SIGNIFICANCE
baseline_phase8.json	–	A frozen run. Every later run is diffed against it, so "did I make it worse" is answerable.
THE CURRENT SCORE 22 / 24 = 91.7% overall. On the 21 unambiguous cases: 20 / 21 = 95.2%. By lane: <code>needs_review</code> 9/9, <code>informational</code> 11/12, <code>safe_auto_fix</code> 2/3. The three ambiguous cases are <i>designed</i> to be arguable — movement there is noise. Movement in the other 21 is signal. That distinction is why two numbers are reported instead of one.		
Configuration		
Dockerfile		One image for both services. Model weights are baked in at build time, so a cold start does not download 90MB from HuggingFace.
docker-compose.yml		app + mcp + postgres + frontend, for local development.
render.yaml		Production as a blueprint. One web service, free plan, <code>autoDeployTrigger: commit</code> .
GUIDE.md		The install guide written for somebody who is <i>not</i> the owner.

SECTION 05

External services — what, why, and what was rejected

Six things this project does not own. For each, an interviewer's real question is "*why that one?*"

GitHub — the platform required

Source of webhooks, logs, diffs; destination for comments. Integrated as a **GitHub App**, not a personal access token.

Why an App: a PAT carries one human's full access to everything they can see, never expires until revoked, and cannot be scoped per repository. An App gets an **installation token scoped to one installation, expiring in an hour by itself**. When Phase 10 leaked a secret, that difference is what limited the blast radius.

Groq — LLM inference required

Runs `openai/gpt-oss-20b`. Called through the **OpenAI SDK**, because Groq is API-compatible.

Why Groq: speed and a free tier. Inference sits inside GitHub's 10-second webhook window (mitigated by backgrounding, but latency still matters). **Why an open-weights 20B rather than a frontier model:** the task is bounded — read an error, name the cause, pick one of three labels. The eval says 95.2% on clear cases. Paying frontier prices for that would be waste.

The trade admitted: the SDK is OpenAI's, so switching providers is a base-URL change. That is deliberate portability, not lock-in.

Neon — Postgres with pgvector required

Stores every diagnosis and the installations table.

Why not Render's own Postgres: Render's free database **expires 30 days after creation**. For most projects that is an annoyance. Here it is fatal — the whole point of the embedding search is recalling failures from weeks ago. *A database that deletes itself every month is not memory*. Neon's free tier does not expire.

Why pgvector rather than a vector database: at this scale the vectors belong next to the rows they describe. A separate Pinecone or Weaviate would add a service, a sync problem, and a second failure mode to solve a problem that `ORDER BY embedding ↔ query` already solves.

Render — hosting required

One free web service running app + MCP server, auto-deploying on push to `master`.

The cost of free: the instance sleeps after ~15 minutes idle and takes 1-2 minutes to wake. **Webhooks that arrive while it sleeps are lost outright** — GitHub does not retry. That was measured, not assumed.

cron-job.org — keep-warm mitigation

Pings `/health` on a schedule so the instance never sleeps and no webhook is lost.

Why not GitHub Actions' own scheduler: it was tried first and **it does not work**. Measured over 7 hours: a `*/10` cron should have fired ~42 times and fired 0. GitHub deprioritises scheduled workflows under load with no error and no notification. An external pinger fires on time. This is in the README as a settled finding.

Discord / Slack webhook — owner notification optional, currently off

One line of text when a private-repo installation needs manual approval.

Currently unset by choice. The message goes to the server log instead. The consequence is stated honestly in the install guide: an installer must message the owner, because nothing else will.

Python dependencies, and what each is load-bearing for

PACKAGE	WHY IT IS THERE
fastapi + uvicorn	Async web framework. Async matters: a webhook handler waiting on an LLM must not block others.
langgraph	The agent as an explicit state machine. Chosen over a prompt-chaining loop because routing becomes inspectable and testable.
mcp	The tool boundary. Makes the write surface enumerable.
sentence-transformers	Local embeddings — no API cost, no third party, no network in the hot path.
pgvector + psycopg + sqlalchemy	Vector column, Postgres driver, schema and queries.
pyjwt[crypto]	Signs the App JWT (RS256). Chosen over a GitHub SDK: the App auth flow is two API calls and a signature — an SDK would be a dependency to hide nine lines.
itsdangerous	Signs the session cookie.
openai	The client for Groq's compatible API.
httpx	Async HTTP throughout.

SECTION 06

The data model

Two tables. Deliberately small.

diagnoses — what happened

COLUMN	HOLDS
id	Primary key
run_id	GitHub's workflow run id — links back to the actual run
repo	owner/name
created_at	Timestamp
log_excerpt	The trimmed log — what the model actually saw
diff_summary	The change that triggered the run
diagnosis_text	The model's explanation
posted_to	Where the comment landed
lane	The classification
embedding	vector(384) — this is what memory searches
raw	The full model response, kept for debugging
installation_id	Which installation produced it — the multi-tenant key

installations — who is allowed

COLUMN	HOLDS
installation_id	GitHub's id for the installation
account_login	User or org name
account_type	User or Organization
is_allowed	The gate. Read on every single webhook
approval_source	auto_public · owner · NULL

WHY APPROVAL_SOURCE HAD TO EXIST

Public-only installations approve themselves. But an install that was all-public can later *add a private repo* — and then automatic approval is a promise that is no longer true, so it must be withdrawn.

The problem: `is_allowed = true` looks identical whether the App set it or a human did. Withdrawing blindly would let the App overrule a decision the owner made on purpose. So the *reason* is recorded. Only `auto_public` is ever auto-withdrawn; `owner` and `NULL` never are.

Rows predating the column are `NULL`, not backfilled to `owner` — because backfilling would write a guess into a column that governs access.

WITHDRAWAL IS ONE-WAY — AND THAT IS THE INTERESTING BIT

Making a repo private withdraws approval. Making it public again does **not** restore it.

Why: if it did, approval would be reversible by anyone who can toggle a repository's visibility. The gate would be controlled by the applicant rather than the approver. Proven live: `privatized` closed the gate; `publicized` arrived three minutes later and did nothing.

Schema changes without Alembic

Migrations are a list of `(description, SQL)` pairs in `db.py`, run at startup, all written `IF NOT EXISTS` so re-running is safe. **Why not Alembic**: at two tables and a handful of additive changes, Alembic is a dependency, a directory, and a mental model to solve a problem this project does not have yet. The trigger to adopt it is the first destructive migration — a column rename or drop — not the calendar.

SECTION 07

The decisions worth defending

This is the section that matters. Each is a decision, a reason, and a rejected alternative.

1 · The 0.90 similarity threshold was derived, not guessed

Memory only helps if "similar" means similar. The threshold was set by measuring real embedding distances between known-related and known-unrelated failure pairs, then picking the value that separated them. Below it, unrelated failures started matching and the hint made diagnoses *worse*.

The honest bit: the eval later found a defect in the untested middle zone around the boundary — and that finding, plus the fix, is written up in the README. A number you can defend with a measurement beats a number that happens to work.

2 · Failing closed, everywhere

- A repository object with no `private` key reads as `private ( .get("private", True) )` — so a payload shape change makes installs wait for a human, not sail through.
- An empty repository list is **no evidence**, and no evidence cannot support a yes.
- An `"All repositories"` install is refused **even when everything is public today**, because it silently covers repos created tomorrow.
- A model response that will not parse falls back to `informational` — the lane that only ever posts a comment.

3 · The field is `private`, not `visibility` — and I checked

GitHub's webhook reference documents that an installation payload contains a `repositories` array, but **not what is inside each object**. Assuming `visibility` — the field the REST API returns — would have shipped a bug that silently made every install look private.

Instead: `GET /app/hook/deliveries`, read a real delivery, and confirm the objects carry exactly five keys — `id, node_id, name, full_name, private`. **Ask the system that owns the fact**. That habit is the single most transferable thing in this project.

4 · Permission changes are asymmetric

Before narrowing `contents` from write to read, the docs were checked rather than assumed. GitHub: *adding* permissions requires every installation to re-accept by hand; *removing* them takes effect immediately. Verified afterwards in practice — both installations moved to `contents: read` with the same `updated_at`, no suspension, no action required from anybody.

5 · The user's GitHub token is thrown away

At the end of OAuth, the access token and its 6-month refresh token are **discarded**. Only the GitHub user id and login go into a signed session cookie.

The cost, stated: the installation list is a snapshot from login, so a newly installed App is invisible until re-login — hence the "Refresh access" button. **The alternative rejected:** storing a live GitHub credential in a signed-but-not-encrypted cookie, or in the same Postgres the dashboard reads. A button press twice a day beats a stored credential.

6 · A secret leaked, and the response is documented

Phase 10 found a leaked secret. It was rotated, the exposure verified closed, and the whole incident written into the README rather than quietly fixed. **An interviewer will respect this far more than a project claiming it never happened.** The reason the blast radius was small is decision 1 in section 5 — scoped, self-expiring installation tokens rather than a personal access token.

7 · Knowing what you have *not* built

The README carries a "Deliberately not done" section: no pagination, no second approver, no refresh-token policy, no scheduled visibility re-check. Each says what would trigger building it — *"the trigger is the row count, not the calendar."*

Being able to name your own gaps, with the condition that would change your mind, reads as engineering judgement rather than an unfinished project.

SECTION 08

Interview questions, with answers

Thirty questions. The **amber line** says what the question is really testing.

Opening — they are checking you can explain your own work

01 Tell me about this project.

Use the pitch in section 1 verbatim. Thirty seconds, then stop and let them steer.

TESTING Whether you can compress. Rambling here costs you the interview.

02 Walk me through what happens when a build fails.

The ten steps in section 2. Emphasise the order: **signature** → **gate** → **respond 200** → **background work**. The reason 200 comes before the work is GitHub's 10-second timeout.

TESTING Do you understand your own control flow, or did you assemble it by trial and error?

03 Why an agent? Could this be a script?

Honest answer: **most of it could**. The agent earns its place at one point — the decision of *which action to take*. Classification and routing are a state machine because the lane determines the action, and that needs to be inspectable and testable rather than buried in prompt text. If it only ever posted comments, a script would do.

TESTING Whether you use "agent" as a buzzword. Conceding the limit is the strong answer.

04 What was the hardest part?

The "repository added to an existing installation later" case. Auto-approval is a promise that every covered repo is public — but repos can be added after. Revoking blindly would overrule a human's deliberate approval; not revoking leaks auto-approval into private repos. Resolved by recording *why* the row was approved (`approval_source`) and only ever auto-withdrawing the App's own decisions.

TESTING Whether you have hit a genuinely hard design problem, or only implementation friction.

Architecture

05 Why MCP? Why not just call the GitHub API?

To make the write surface **enumerable**. Every action that touches GitHub goes through one described tool call, so the complete list of what this agent can do is four signatures. Without that boundary, "what can it do to my repo" is answered by reading the whole codebase.

TESTING Whether you chose a technology or followed a trend.

06 Why LangGraph over LangChain or a plain loop?

Explicit state and explicit routing. The lane decides the next node through a pure function, so control flow is a graph you can read. A prompt-chaining loop hides routing inside the model's output, which is exactly the part you most need to test.

TESTING Framework literacy, and whether you can articulate a trade rather than a preference.

07 Your MCP server runs in the same process. Isn't the separation fake?

Good question — the process boundary is fake, the code boundary is not. Under Docker they are separate containers. On Render's single free service they were merged by `server_combined.py`. The enforcement that matters — the agent reaching GitHub only through described tools — is unchanged either way. Paying for a second service to make a diagram honest would be the wrong trade.

TESTING Whether you defend a design under pressure or fold. Concede the real point, hold the substantive one.

08 How do you stop it re-running a job forever?

GitHub's own `run_attempt`. Only attempt 1 is ever re-run. Because the counter is GitHub's rather than in-process state, a restart or a redeploy cannot reset it.

TESTING Do you understand why stateless guards beat stateful ones?

09 What happens if the LLM returns garbage?

Structured output is requested and validated. An unparseable or invalid response falls back to `FALLBACK_CATEGORY = informational` — the lane whose only action is posting a comment. The failure mode is a less useful comment, never an unintended write.

TESTING Whether you designed for the model being wrong.

10 How would this scale to a thousand repositories?

Three things break first, in order: (1) the free Render instance — webhook processing is in-process background tasks, so this needs a real queue (Celery, or Postgres-backed jobs); (2) `/api/diagnoses` uses a `LIMIT`, not pagination, and silently truncates; (3) the pgvector search is unindexed — it needs an HNSW index, which is one statement.

TESTING Can you find your own bottlenecks? Naming three in order is much better than "add caching".

Security and access — usually the deepest questions

11 How do you authenticate to GitHub?

As a GitHub App. Sign a JWT with the App's private key (RS256), trade it for an **installation access token** scoped to one installation and expiring in one hour. Cached in memory for its lifetime. No personal access token anywhere in the system.

TESTING Do you actually understand the auth flow you implemented?

12 Why not a personal access token? It's simpler.

A PAT carries one human's full access to everything they can see, never expires until revoked, and cannot be scoped per repository. An installation token is scoped and self-expiring. When a secret leaked in Phase 10, that difference is what bounded the damage.

TESTING Security reasoning about blast radius, not just "tokens are bad".

13 How do you verify a webhook is really from GitHub?

HMAC-SHA256 over the **raw request body** using the shared secret, compared with `hmac.compare_digest` — constant-time, so it does not leak the signature through timing. It runs before parsing, because parsing attacker-controlled JSON before authenticating it is the bug.

TESTING Whether you know *why* constant-time comparison and ordering matter.

14 Who can approve an installation?

Only the account that registered the App, read from GitHub's `GET /app` at request time — not from a config file or an env var that could drift out of sync with reality.

TESTING Authoritative-source thinking again.

15 You auto-approve public repos. Isn't that a security hole?

No, and the reason is the whole argument: **a public repository's build logs are already readable by anyone**. BuildDoctor reading one discloses nothing that was not already open, so requiring approval would be permitting something already permitted. Private logs are a genuine decision, so a human still makes it. The rule is narrow — it needs an explicitly selected repo list *and* every repo public.

TESTING Can you reason about what a control actually protects, rather than adding controls reflexively?

16 Why refuse "All repositories" if everything is public today?

Because approval is a **promise about tomorrow**. An "all" install covers every repo the account creates from then on. GitHub's own community thread contradicts itself on whether `installation_repositories` fires for repos created later under an "all" install — so the promise cannot be kept, and it is refused rather than guessed at.

TESTING Whether you act on ambiguous evidence by failing closed.

17 Someone makes their repo private after being auto-approved. Then?

The `repository` / `privatized` webhook fires and approval is withdrawn — but only if `approval_source = auto_public`. An owner-approved install is never auto-withdrawn, because that would be the App overruling a deliberate human decision.

TESTING Did you think past the happy path?

18 They make it public again. Does approval come back?

No — **deliberately**. If it did, approval would be reversible by anyone who can toggle a repository's visibility, which puts the gate in the applicant's hands rather than the approver's.

TESTING Whether you can spot a privilege-escalation path in your own design.

19 Where do you store the user's GitHub token?

Nowhere. It is discarded at the end of the OAuth callback along with the refresh token. The session cookie holds the GitHub user id and login, signed with `itsdangerous`.

TESTING Session design, and whether you know what you are trading away.

20 Your admin endpoint has no CSRF token. Why is that acceptable?

Fair challenge. The mitigation is that the route requires a valid session *and* that the caller is the App owner, and it was verified that an unauthenticated POST with a well-formed body returns **401 with the row unchanged**. It is not a substitute for CSRF protection on a larger surface — with one write route and one authorised human, it is a stated, bounded gap rather than an oversight.

TESTING How you handle a real weakness. Defending it as flawless is the wrong answer; naming the bound is the right one.

21 Your MCP server is on a public URL. Exposed?

Reachable, not usable. `mcp_server.py` checks the `Host` header against an allowlist and returns **421** on a mismatch. Traffic through Render's router carries the public hostname, which is not listed; the app's own loopback call carries `localhost:10000`, which is. Tested in both directions.

TESTING DNS-rebinding awareness — a genuinely advanced answer.

Data, memory, and evaluation

22 Explain the memory feature.

Each log excerpt is embedded to 384 dimensions with `all-MiniLM-L6-v2`, stored in a pgvector column. On a new failure, a cosine search finds the nearest past diagnosis; above **0.90** similarity it is passed to the model as a hint. It is retrieval-augmented generation over the project's own history.

TESTING Whether you can explain embeddings without hand-waving.

23 Where did 0.90 come from?

Measurement. Distances were compared between known-related and known-unrelated failure pairs, and the threshold set where they separated. Below it, unrelated failures matched and the hint made diagnoses worse. The eval later found a defect in the untested zone near the boundary, which was fixed — that is written up rather than hidden.

TESTING The single best question to be ready for. "It felt right" fails. A measurement passes.

24 Why a local embedding model instead of an API?

Cost, latency, and disclosure. It is free per call, has no network hop in the hot path, and the log is not sent to a second third party. The weights are baked into the Docker image so a cold start does not pull 90MB from HuggingFace.

TESTING Practical cost and latency reasoning.

25 How do you know the classifier is any good?

A 24-case golden set with known-correct lanes, in four groups including a deliberately ambiguous "middle zone". Current: **91.7% overall, 95.2% on unambiguous cases**. It calls the real classifier — not a copy — but never routes to the action nodes, so running it posts nothing.

TESTING Whether you evaluate at all. Most candidate projects do not, and this answer stands out sharply.

26 Why report two accuracy numbers?

Because three of the 24 cases are *designed* to be arguable. Movement on those is noise; movement on the other 21 is signal. Reporting one blended number would hide the distinction that makes the metric useful.

TESTING Metric literacy. A strong differentiator.

27 Why Postgres + pgvector rather than a vector database?

The vectors describe rows that already live in Postgres. A dedicated vector store would add a service, a sync problem, and a second failure mode to solve something one `ORDER BY` already solves. The trigger to revisit is scale — and the first fix would be an HNSW index, not a new service.

TESTING Resisting resume-driven architecture.

Operations

28 What happens if your server is asleep when a webhook arrives?

It is lost. GitHub does not retry. That was measured, not assumed. The fix is an external pinger keeping the instance warm. GitHub Actions' own scheduler was tried first and does not work — over 7 hours a `*/10` cron should have fired ~42 times and fired 0, silently.

TESTING Do you know your system's real failure modes? Admitting lost webhooks is far stronger than claiming reliability you cannot prove.

29 How do you know it is working right now?

`python preflight.py` — ten checks in about thirty seconds. It asks Render, Neon, Groq and GitHub directly, verifies every installation's gate state, confirms the deployed bundle, checks recent webhook delivery outcomes, and asserts the protected routes still return 401. It exists because free-tier dependencies fail silently and uptime proves nothing about the others.

TESTING Operational maturity. Very few candidate projects have this.

30 What would you do next?

In order: (1) a real job queue so webhook processing survives a restart; (2) pagination before the row count makes `LIMIT` dishonest; (3) a scheduled visibility re-check, because withdrawal currently depends on webhooks arriving and a sleeping server loses them; (4) an HNSW index when the search gets slow.

Each has a stated trigger. None is "rewrite it in Rust".

TESTING Prioritisation. Triggers rather than a wish list is the difference.

SECTION 09

Proof it works

Captured 2026-08-28. Not a description of what should happen — a record of what did.

The end-to-end run

```
16:01:09  commit 78e7707 pushed to builddoctor-testing/ci-test
16:01:2x  workflow run 33187951570 → failure
16:01:38  comment posted by builddoctor-ci[bot]
16:01:39  diagnoses row id=12 written, embedding stored

elapsed: ~18 seconds, build failure to comment
```

What the bot actually wrote:

```
## BuildDoctor: `CI` failed

The job failed because a step in the workflow ran the command
`exit 1`, which immediately caused the job to terminate with exit
code 1. The added `trigger.txt` file is unrelated and does not
explain the failure.

Failed job(s): build
Lane: informational - STEP 2 (build or dependency machinery):
The failure originates from a workflow step (`Run exit 1`)
defined in the CI configuration, not from application code.
```

WHY THIS OUTPUT IS WORTH SHOWING AN INTERVIEWER

It did not just read the error — it **ruled out the diff**. The commit added `trigger.txt`, and the model explicitly said that file is unrelated. That is the diff-gathering step in section 2 doing real work; a log-only tool could not have said it.

Preflight, same session

```
[ ok ] environment: all 8 keys present
[ ok ] render: not suspended; deploy live
[ ok ] server: awake, /health in 0.58s
[ ok ] dashboard: serving bundle index-0tVG-ofB.js
[ ok ] database: 3 installs, all open
[ ok ] groq: key valid, openai/gpt-oss-20b available
[ ok ] github app: builddoctor-ci, 3 installs, perms and events correct
[ ok ] webhooks: last 20 delivered ok
[ ok ] auth gate: all 3 routes return 401
[ ok ] git: clean and pushed
-----
10 pass, 0 warn, 0 fail
```


Real multi-tenant traffic

ROW	REPO	INSTALLATION	LANE
12	builddoctor-testing/ci-test	157254438	informational
11	Dhatubablu/samplerepofirst	157269524	informational
10	Dhatubablu/samplerepofirst	157269524	informational
9	builddoctor-testing/ci-test	157254438	informational

Rows 10 and 11 come from an installation that is **not the author's**. Multi-tenancy is demonstrated by traffic, not asserted by design.

IF YOU SHOW ONE THING IN AN INTERVIEW, SHOW THIS

Open the commit, show the bot's comment, then open the dashboard and show the same event as a row. Two artefacts, one event, thirty seconds. That sequence proves the whole pipeline without a single slide.

BuildDoctor — complete technical reference. Generated 2026-08-28.
Every figure in this document was read from the running system or the source, not recalled.